Estructuras de Datos y Algoritmos

Tema 10: Árboles binarios de búsqueda

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Propiedad de búsqueda

Un Arbol Binario de Busqueda (ABB) añade una restricción sobre los datos al árbol binario que ya conocemos.

Propiedad global

Para cada nodo x:

- Si y pertenece al subárbol izquierdo de x, entonces $y.\mathrm{dato} \leq x.\mathrm{dato}$.
- Si y pertenece al subárbol derecho de x, entonces $y.\mathrm{dato} \geq x.\mathrm{dato}$.

La condición alcanza a **todos los nodos** de cada subárbol. Se permiten datos repetidos.

La propiedad permite buscar y obtener mínimos, máximos, sucesores y predecesores. Insertar y eliminar deben conservarla.

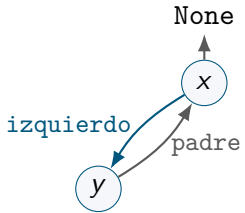
Representación

Conservamos `NodoBinario` y añadimos `padre`.

```
class NodoBinario:
    def __init__(self, dato):
        self.dato = dato
        self.izquierdo = None
        self.derecho = None
        self.padre = None

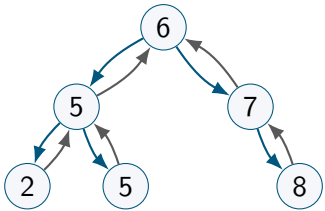

class ArbolABB:
    def __init__(self):
        self.raiz = None
```

`padre` permite ascender. `ArbolABB` permite actualizar `T.raiz` al insertar o eliminar.

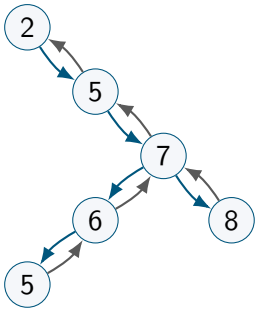


Azul: referencia al hijo. Gris: referencia a `padre`.

Dos formas



Seis nodos, altura 2.



Seis nodos, altura 4.

Ambos contienen $2, 5, 5, 6, 7, 8$. La forma cambia la altura y el tiempo de sus operaciones.

Inorden en un ABB

Inorden: subárbol izquierdo, nodo, subárbol derecho.

Al visitar x, los datos de su subárbol izquierdo son $\leq x.\mathrm{dato}$. Después se visitan los del subárbol derecho, que son $\geq x.\mathrm{dato}$. Como la propiedad se cumple en cada nodo, el recorrido produce un orden no decreciente.

Salida en ambos árboles

2, 5, 5, 6, 7, 8.

El recorrido conserva el costo $\Theta(n)$ (visto en el tema anterior).

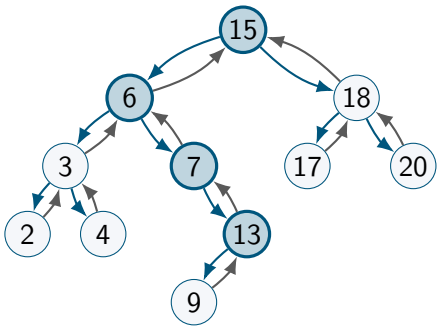
Búsqueda

Para buscar 13, seguimos el camino:

$$15 \longrightarrow 6 \longrightarrow 7 \longrightarrow 13.$$

- $13 < 15$: hijo izquierdo.
- $13 > 6$: hijo derecho.
- $13 > 7$: hijo derecho.
- $13 = 13$: retornar este nodo.

Cada comparación descarta uno de los subárboles.



Búsqueda recursiva

Dados `nodo` y k, retorna un nodo del subárbol con dato k, o `None`.

Función Buscar(nodo, k)

Si nodo es vacío o $k =$ nodo.dato:
    Retornar nodo
Si $k <$ nodo.dato:
    Retornar Buscar(nodo.izquierdo, k)
Si no:
    Retornar Buscar(nodo.derecho, k)

```
def buscar(nodo, k):
    if (nodo is None
            or k == nodo.dato):
        return nodo
    if k < nodo.dato:
        return buscar(
            nodo.izquierdo, k)
    else:
        return buscar(
            nodo.derecho, k)
```

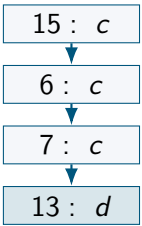
Para buscar en todo el árbol: `buscar(T.raiz, k)`.

Costo de búsqueda

Sea r el número de descensos recursivos hasta encontrar k o llegar a `None`.

$$T(0) = d, \qquad T(r) = T(r-1) + c, \quad r \geq 1.$$

Llamadas al buscar 13



Tres descensos, cuatro llamadas:

$$T(3) = 3c + d.$$

Expansión para $r \geq 2$

$$\begin{aligned} T(r) &= T(r-1) + c \\ &= T(r-2) + 2c \\ &\vdots \\ &= T(r-r) + rc = d + rc. \end{aligned}$$

En un árbol no vacío de n nodos y altura h, se cumple $r \leq h + 1$

$$T(r) \leq c(h+1) + d \quad \Longrightarrow \quad \boxed{T(r) \in O(h+1).}$$

Búsqueda iterativa

El ciclo sigue el mismo camino que la versión recursiva.

Función BuscarIterativo(nodo, k)

Mientras nodo no es vacío y $k \neq$ nodo.dato:
    Si $k <$ nodo.dato:
        nodo $\leftarrow$ nodo.izquierdo
    Si no:
        nodo $\leftarrow$ nodo.derecho
Retornar nodo

```
def buscar_iterativo(nodo, k):
    while (nodo is not None
           and k != nodo.dato):
        if k < nodo.dato:
            nodo = nodo.izquierdo
        else:
            nodo = nodo.derecho
    return nodo
```

Retorna una referencia al nodo encontrado o `None`.

Mínimo y máximo

Función Minimo(nodo)

Mientras nodo.izquierdo no es vacío:
    nodo ← nodo.izquierdo

Retornar nodo

```
def minimo(nodo):
    while (nodo.izquierdo
           is not None):
        nodo = nodo.izquierdo
    return nodo
```

Función Maximo(nodo)

Mientras nodo.derecho no es vacío:
    nodo ← nodo.derecho

Retornar nodo

```
def maximo(nodo):
    while (nodo.derecho
           is not None):
        nodo = nodo.derecho
    return nodo
```

Costo. Cada función sigue un solo camino: $O(h+1)$, donde h es la altura del subárbol inicial.

Sucesor

Sucesor de un nodo

Es el **siguiente nodo visitado en un recorrido inorden**.

Si todos los datos son distintos, el sucesor de x tiene el menor dato mayor que $x.\mathrm{dato}$.

Con datos repetidos, el sucesor puede tener el mismo dato que x. La definición identifica el siguiente **nodo**.

Si x es el último nodo del recorrido, su sucesor es `None`. La estructura del árbol permite encontrarlo sin comparar datos.

Subárbol derecho

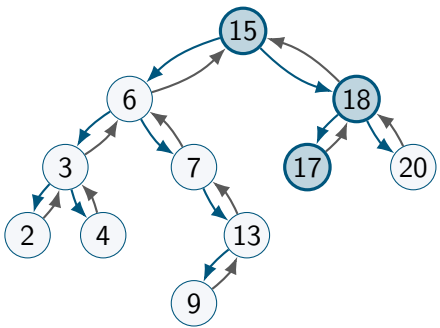
Si el subárbol derecho de x existe, su sucesor es el nodo más a la izquierda de ese subárbol:

`Minimo(x.derecho)`.

Para el nodo con dato 15:

$$15 \longrightarrow 18 \longrightarrow 17.$$

El resultado es el nodo con dato 17.

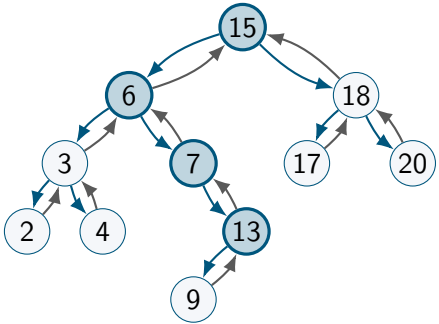


Ascender por padres

Si x no tiene subárbol derecho, buscamos el ancestro más cercano para el cual x pertenece al subárbol izquierdo.

Ascendemos mediante `padre`:

$$13 \longrightarrow 7 \longrightarrow 6 \longrightarrow 15.$$



- 13 es hijo derecho de 7.
- 7 es hijo derecho de 6.
- 6 es hijo izquierdo de 15.

El sucesor de 13 es el nodo con dato 15.

Algoritmo del sucesor

Función Sucesor(nodo)

Si nodo.derecho no es vacío:
    Retornar Minimo(nodo.derecho)

Si no:
    $y \leftarrow$ nodo.padre
    Mientras y no es vacío y nodo es hijo derecho de y:
        nodo $\leftarrow y$
        $y \leftarrow y$.padre
    Retornar y

El segundo caso asciende mientras el nodo actual sea hijo derecho. Retorna el primer padre para el cual el nodo original queda en su subárbol izquierdo, o `None` si no existe.

Sucesor en Python

Precondición: `nodo` es un nodo existente del árbol.

```
def sucesor(nodo):
    if nodo.derecho is not None:
        return minimo(nodo.derecho)
    else:
        y = nodo.padre
        while y is not None and nodo is y.derecho:
            nodo = y
            y = y.padre
        return y
```

`nodo is y.derecho` compara la identidad de los nodos, no sus datos. La función retorna una referencia al sucesor o `None`.

Costo. $O(h+1)$, con h la altura del árbol: se sigue un camino descendente o ascendente.

Predecesor

Predecesor de un nodo

Es el nodo anterior en el recorrido inorden.

Se obtiene de forma simétrica al sucesor:

- Si el subárbol izquierdo existe, tomar su nodo más a la derecha mediante `Maximo(nodo.izquierdo)`.
- Si no existe, ascender hasta el ancestro más cercano para el cual x pertenece al subárbol derecho.

Si no hay un nodo anterior, el resultado es `None`.

Actividad

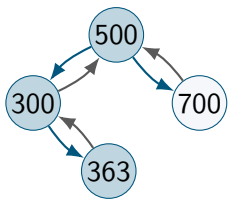
Buscamos 363 desde la raíz de un ABB con datos entre 1 y 1000.

Cada secuencia lista los datos visitados: primero la raíz y, después, el hijo al que desciende la búsqueda en cada paso.

Analiza cada opción por separado; no se supone un mismo árbol para todas.

a. 2, 252, 401, 398, 330, 344, 397, 363.
b. 924, 220, 911, 244, 898, 258, 362, 363.
c. 925, 202, 911, 240, 912, 245, 363.
d. 2, 399, 387, 219, 266, 382, 381, 278, 363.
e. 935, 278, 347, 621, 299, 392, 358, 363.

Ejemplo independiente



Camino: $500 \to 300 \to 363$.
El nodo 700 no se visita.

¿Qué secuencias son imposibles? Justifica en qué paso se viola la propiedad de búsqueda.

Respuesta

Al buscar 363, cada comparación restringe los datos que pueden aparecer después en el camino.

Caso	Resultado	Restricción que se viola
c	Imposible	Después de visitar 911, la búsqueda baja a su subárbol izquierdo. El siguiente $912 > 911$ queda fuera de ese subárbol.
e	Imposible	Después de visitar 347, la búsqueda baja a su subárbol derecho. El posterior $299 < 347$ queda fuera de ese subárbol.

Las secuencias **a, b y d son posibles**: sus datos respetan los límites acumulados durante la búsqueda.

Inserción

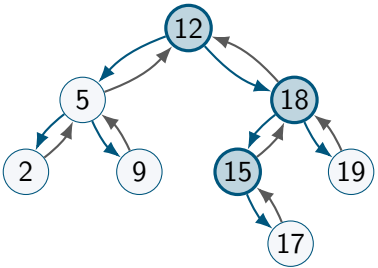
`INSERTAR(T,nodo)` recibe el árbol T y un **nodo nuevo**, `nodo`.

- `nodo.dato` ya contiene la clave que se insertará.
- `nodo.izquierdo = None` y `nodo.derecho = None`.
- El procedimiento modifica T y algunos atributos de `nodo` para colocarlo en una posición adecuada.

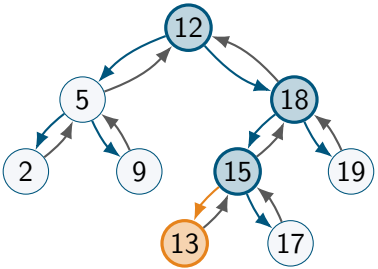
La función `insertar(T, nodo)` recibe ese mismo nodo, modifica los enlaces y devuelve `None` implícitamente.

Ejemplo de inserción

Antes de insertar



Después de insertar



Las comparaciones llevan por 12, 18 y 15. El nuevo nodo 13 ocupa la posición `None` del hijo izquierdo de 15.

Algoritmo de inserción

Procedimiento Insertar(T, nodo)

1. Descender

actual $\leftarrow$ T.raiz
$y \leftarrow$ vacío
Mientras actual no es vacío:
    $y \leftarrow$ actual
    Si nodo.dato $<$ actual.dato:
        actual $\leftarrow$ actual.izquierdo
    Si no:
        actual $\leftarrow$ actual.derecho

2. Conectar

nodo.padre $\leftarrow$ y
Si y es vacío:
    T.raiz $\leftarrow$ nodo
Si no, si nodo.dato $<$ y.dato:
    y.izquierdo $\leftarrow$ nodo
Si no:
    y.derecho $\leftarrow$ nodo

Primero se localiza una posición vacía; después se conecta `nodo`.

Inserción en Python

1. Descender por el árbol

```
def insertar(T, nodo):
    actual = T.raiz
    y = None
    while actual is not None:
        y = actual
        if nodo.dato < actual.dato:
            actual = actual.izquierdo
        else:
            actual = actual.derecho
```

`nodo` conserva la referencia al objeto nuevo. `actual` recorre el árbol y *y* guarda el nodo examinado antes de cada descenso.
Si las claves son iguales, la rama `else` continúa por la derecha.

2. Conectar el nuevo nodo

Continuación de `insertar(T, nodo)`:

```
    nodo.padre = y
    if y is None:
        T.raiz = nodo
    elif nodo.dato < y.dato:
        y.izquierdo = nodo
    else:
        y.derecho = nodo
```

Al terminar el ciclo, `actual` es `None` y y indica el padre de `nodo`. Si y también es `None`, el árbol estaba vacío y `nodo` será su raíz.

En un árbol no vacío de altura h, el descenso realiza como máximo $h + 1$ iteraciones de costo constante. Conectar `nodo` cuesta $\Theta(1)$. Por tanto,

$$\boxed{T_{\text{insertar}} \in O(h + 1).}$$

Eliminación

`eliminar(T, nodo)` recibe un nodo **ya localizado** en T. En las figuras, ese nodo se identifica como z.

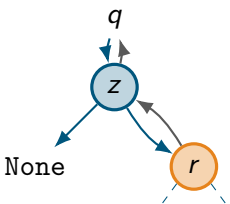
Trasplante

`trasplantar(T, nodo, v)` reemplaza el enlace hacia `nodo` (del padre o de `T.raiz`) por un enlace hacia v, que puede ser `None`. Si v existe, su `padre` pasa a ser el padre de `nodo`.
No modifica los hijos de `nodo` ni de v.

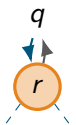
1. **Sin hijos.** Trasplantar z por `None`.
2. **Con un hijo.** Trasplantar z por su hijo.
3. **Con dos hijos.** Trasplantar z por su sucesor y y conectar los subárboles de z a y.

Sin hijo izquierdo

Antes



Después



Trasplante: `trasplantar(T, z, r)`. El enlace hacia z pasa a r; si r existe, se actualiza `r.padre`.
q representa al padre de z, si existe. Si z es la raíz, se actualiza `T.raiz`. Si $r =$ `None`, z era una hoja y esa posición queda vacía.

Sin hijo derecho



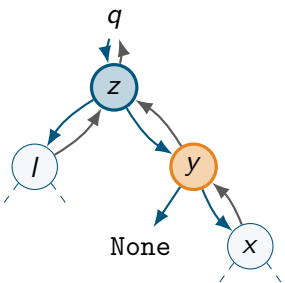
Trasplante: `trasplantar(T, z, l)`. El enlace hacia z pasa a l, y `l.padre` recibe el padre de z.

Aquí l existe y `z.derecho` es `None`. Se conserva todo el subárbol de l.

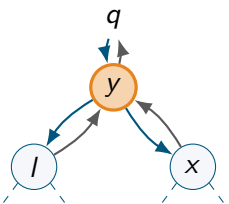
Dos hijos

El sucesor es `y = minimo(z.derecho)`.

Antes: $y =$ `z.derecho`



Después

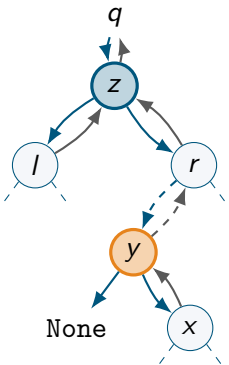


1. **Trasplante:** `trasplantar(T, z, y)`. Se actualiza el enlace hacia z y `y.padre`.
2. **Conexión adicional:** `y.izquierdo = l` y `l.padre = y`.

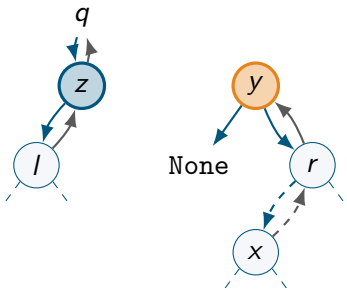
y no tenía hijo izquierdo y conserva su hijo derecho x, si existe.

Sucesor más profundo

Antes: $y \neq$ `z.derecho`



Esquema intermedio

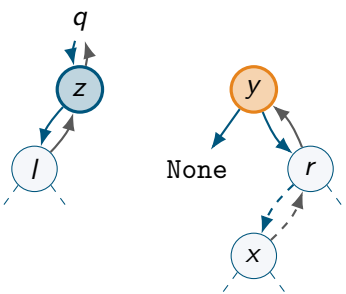


1. **Trasplante:** `trasplantar(T, y, x)`, con $x =$ `y.derecho`.
2. **Conexión adicional:** `y.derecho = r` y `r.padre = y`.

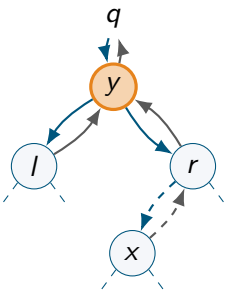
x puede ser `None`. El esquema omite `z.derecho = r` y el antiguo `y.padre`; los caminos discontinuos omiten nodos intermedios.

Reubicar el sucesor

Tras los pasos 1 y 2



Resultado final



3. Trasplante: `trasplantar(T, z, y)`. Se actualiza el enlace hacia z y `y.padre`.

4. Conexión adicional: `y.izquierdo = l` y `l.padre = y`.

Ahora y ocupa la posición de z, con subárboles izquierdo l y derecho r.

Trasplante

Conecta v en el lugar de `nodo`. Si v es `None`, deja ese enlace vacío.

Procedimiento Trasplantar(T, nodo, v)

Si nodo.padre es vacío:
    T.raiz $\leftarrow v$
Si no, si nodo es hijo izquierdo:
    nodo.padre.izquierdo $\leftarrow v$
Si no:
    nodo.padre.derecho $\leftarrow v$
Si v no es vacío:
    v.padre $\leftarrow$ nodo.padre

```
def trasplantar(T, nodo, v):
    if nodo.padre is None:
        T.raiz = v
    elif nodo is
    nodo.padre.izquierdo:
        nodo.padre.izquierdo = v
    else:
        nodo.padre.derecho = v
    if v is not None:
        v.padre = nodo.padre
```

No cambia los hijos de `nodo` ni de v. **Costo.** $\Theta(1)$: solo comprueba condiciones y modifica referencias.

Algoritmo de eliminación

`nodo` ya está localizado y corresponde a z en las figuras.

Procedimiento Eliminar(T, nodo)

Cero o un hijo

Si nodo.izquierdo es vacío:
    Trasplantar(T, nodo, nodo.derecho)

Si no, si nodo.derecho es vacío:
    Trasplantar(T, nodo, nodo.izquierdo)

Dos hijos

Si no:
    $y \leftarrow$ Minimo(nodo.derecho)
    Si y no es nodo.derecho:
        Trasplantar(T, y, y.derecho)
        y.derecho $\leftarrow$ nodo.derecho
        y.derecho.padre $\leftarrow y$
    Trasplantar(T, nodo, y)
    y.izquierdo $\leftarrow$ nodo.izquierdo
    y.izquierdo.padre $\leftarrow y$

Con dos hijos, y es el sucesor que ocupará la posición de `nodo`.

Eliminación en Python

1. Elegir el caso y localizar el sucesor

```
def eliminar(T, nodo):
    if nodo.izquierdo is None:
        trasplantar(T, nodo, nodo.derecho)
    elif nodo.derecho is None:
        trasplantar(T, nodo, nodo.izquierdo)
    else:
        y = minimo(nodo.derecho)
        if y is not nodo.derecho:
            trasplantar(T, y, y.derecho)
```

Con cero o un hijo, el trasplante completa la eliminación. Con dos hijos, `minimo` obtiene el sucesor y.
Si y está más abajo que `nodo.derecho`, primero su hijo derecho ocupa su posición anterior.
La comparación `is not` distingue nodos, no sus datos.

Eliminación en Python

2. Reubicar el sucesor y conectar los subárboles

Continuación de `eliminar(T, nodo)`:

```
            y.derecho = nodo.derecho
            y.derecho.padre = y
        trasplantar(T, nodo, y)
        y.izquierdo = nodo.izquierdo
        y.izquierdo.padre = y
```

Las primeras dos asignaciones siguen dentro de `if y is not nodo.derecho`: solo el sucesor profundo recibe el subárbol derecho de `nodo`.

Después, en ambos casos con dos hijos, *y* ocupa la posición de `nodo` y recibe su subárbol izquierdo. Cada conexión actualiza también `padre`.

La función modifica el árbol y devuelve `None` implícitamente.

Costo e identidad

Suponemos que `nodo` ya está localizado; no incluimos su búsqueda.

- **Cero o un hijo.** Un trasplante: $\Theta(1)$.
- **Dos hijos.** `minimo` sigue un solo camino; las reconexiones cuestan $\Theta(1)$.

En el caso con dos hijos:

$$T_{\text{eliminar}} = T_{\text{minimo}} + \Theta(1) \in O(h+1),$$

donde h es la altura del árbol antes de eliminar.

Se retira **exactamente el objeto `nodo`**. Con dos hijos, el mismo objeto y ocupa su posición: no se copian los datos de y a `nodo`. Los demás nodos conservan su identidad.